

BARA: Adaptive Workflow Ranking and Budget Allocation for Code Generation

Anonymous Author(s)

Abstract—Most code-generation evaluations treat each task as a one-workflow problem, fixing a single inference procedure in advance. The same task, however, can now be attempted through a growing set of inference workflows that take different solution paths, succeed on different tasks, and cost different amounts; no single choice fits every task. Committing to one workflow then misses tasks another would solve, yet running them all wastes tokens on poorly ordered or over-budget attempts. No existing setting provides a principled way to order candidate workflows and allocate budget across attempts under a fixed per-task token budget. In this study, we take a first step toward bridging this gap by modeling code generation as ranked workflow execution with task-level budget allocation, where the system ranks candidate workflows and assigns token caps across successive attempts. We instantiate this setting with three inference workflows that capture distinct code-generation patterns: direct coding, plan-structured coding, and test-guided repair. We introduce BARA, an adaptive framework that ranks workflows with a nearest-neighbor policy and estimates workflow-level token caps from historical cost evidence on similar tasks. Across two open-weight code models, one proprietary LLM, and three budget regimes, BARA improves solve rate over the best individual workflow by up to 7.1% and outperforms repeated sampling in eight of nine settings. It also raises first-hit rate by up to 4.1% over the uniform ranking baseline. Adaptive allocation reduces capped-attempt rate for every model-budget pair by up to 16.8%, and a token-aware reward lowers average inference cost by up to 2.45 thousand accounted tokens per task without reducing solve rate. Robust across benchmarks, these results frame multi-workflow code generation as a budgeted orchestration problem beyond the choice between a fixed workflow and exhaustive execution.

Index Terms—Large Language Models, Code Generation, Budgeted Inference

I. INTRODUCTION

Code generation has long been handled as a one-workflow problem, where each programming task is solved by a single fixed inference procedure [1], [2]. Fixing the workflow in advance turns it into a global design choice, applied uniformly to every task regardless of its specification structure, difficulty, or failure mode. This assumption has become limiting, because the same task can now be attempted through several inference workflows that follow different solution paths, solve different subsets of tasks, and incur different token costs [3], [4], [5], [6]. Relying on a single workflow then misses tasks solvable only by another, while running every workflow on every task increases coverage only at the cost of unnecessary computation and latency.

When multiple attempts share one budget, an expensive failed attempt can also prevent later complementary workflows from being tried. The central gap is therefore the absence of a principled mechanism for using multiple workflows under

a finite task-level budget [7]. Existing settings typically treat the workflow either as a fixed choice made before execution or as an unstructured set of attempts run without adaptive control [8], [9], [10], leaving open which workflow to attempt first, how much budget each attempt receives, and how execution proceeds after a failure.

This paper studies code generation as ranked workflow execution with task-level budget allocation, a sequential decision problem over successive attempts. Given a task, a finite token budget, and a set of candidate workflows, the system ranks the workflows, assigns a token cap to each successive attempt, and stops once an attempted solution is accepted by the evaluator. The ranking determines the fallback order, while the allocation constrains how much budget each workflow may consume before control passes to the next candidate. This formulation extends single-action orchestration by producing an ordered fallback sequence and distributing one task-level budget across distinct executable workflows, preserving the benefit of fallbacks without requiring exhaustive execution [11].

The proposed framework BARA learns ranking and allocation from prior executions on similar tasks. Its ranker estimates task-conditioned workflow utility from sparse historical outcomes with a nearest-neighbor ranking policy [11], [12]. Its allocator estimates workflow completion costs from neighboring executions, including capped attempts whose true costs are right-censored, and converts these estimates into per-attempt token caps. After a failed or capped attempt, the remaining budget passes to the next workflow. Once a solution is accepted, execution stops. Under this execution model, each workflow acts as a budgeted action whose value depends on both its probability of solving the task and the computation it leaves for later alternatives [13], [14].

We instantiate the framework with three representative inference workflows. *Direct Coding* produces final code in a single model call. *Plan-Structured Coding* adds a concise implementation checklist before the code, within the same response [15], [16]. *Test-Guided Repair* jointly drafts code and self-tests, and repairs only if the self-tests fail [17], [18], [19], [20]. These workflows differ in reasoning structure, execution cost, and access to feedback, solving overlapping yet complementary sets of tasks.

We evaluate BARA on the full BigCodeBench benchmark, which spans both easier programming tasks and harder specifications with richer library and behavioral constraints [21], and assess generalization on LiveCodeBench (v6), a suite of recent competitive-programming problems [22]. Experiments cover

two open-weight code models and the proprietary Claude Opus-4.8 across three budget regimes from scarce to largely sufficient. BARA improves solve rate over the best individual workflow by up to 7.1% and outperforms repeated sampling in eight of nine settings. First-hit rate rises by up to 4.1% over the uniform ranking baseline, while adaptive allocation lowers the capped-attempt rate for every model-budget pair by up to 16.8%. Token-aware feedback further reduces mean inference cost by up to 2.45 thousand accounted tokens per task while preserving solve rate. On LiveCodeBench, BARA again outperforms every individual workflow across budgets, generalizing the gains to a distinct task distribution. This paper makes the following contributions:

- A formulation of multi-workflow code generation as ranked workflow execution with task-level budget allocation, shifting the problem from selecting one fixed workflow to coordinating workflow order, budget allocation, and stopping decisions under a finite token budget.
- An adaptive orchestration framework that ranks workflows and estimates per-workflow token caps from historical cost evidence on similar tasks, explicitly modeling the right-censored costs of capped attempts.
- An empirical study across three models and three budget regimes, with a complementary workflow set covering Direct Coding, Plan-Structured Coding, and Test-Guided Repair, showing that adaptive orchestration raises solve rate and first-hit rate while token-aware feedback reduces inference cost without sacrificing correctness.

II. BACKGROUND AND RELATED WORK

A. Large Code Models and Code-Generation Benchmarks

Chen *et al.* [1] introduced Codex and HumanEval, establishing executable functional correctness and pass@k as standard evaluation criteria for code generation, while Austin *et al.* [23] introduced MBPP for few-shot and fine-tuning evaluation. Open code models were scaled through large-scale sampling, multi-turn synthesis, pretraining, and instruction tuning [24], [25], [26], [27], [28], [29], [30]. Evaluation then moved toward more realistic and complex programs, from data-science code in DS-1000 [31] to library-intensive tasks in BigCodeBench [21] and competitive-programming problems in LiveCodeBench [22]. As task heterogeneity grows, no single fixed workflow covers the full range of solvable tasks.

B. Inference-Time Workflows for Code Generation

Inference-time methods improve generation by adding reasoning, sampling, execution feedback, or refinement. Chain-of-thought prompting, self-consistency, Tree-of-Thoughts, and ReAct use intermediate reasoning paths or action interleaving to improve model outputs [15], [32], [33], [34]; Structured Chain-of-Thought, PAL, program-of-thought prompting, and LEVER adapt these ideas to code generation and execution-based reranking [16], [35], [36], [37]. Repair-oriented systems such as Self-Debugging, Self-Refine, Reflexion, and Open-CodeInterpreter use execution feedback or verbal feedback to revise incorrect programs [18], [19], [38], [20], [39], [40].

Recent workflow systems compose or adapt LLM invocation structures, including QualityFlow and AdaptFlow [41], [42]. Different workflows improve code generation through different mechanisms while incurring different costs and failure modes.

C. Resource-Aware Selection and Budgeted Inference

Resource-aware selection studies how to choose among candidate solvers, models, or inference procedures when quality and cost vary by input. Recent LLM-routing systems dynamically dispatch queries to different models or cascades to trade quality for cost [43], [44], [45]. Nearest-neighbor methods provide a lightweight way to transfer evidence across similar inputs without fitting a global predictor; recent work uses kNN retrieval for in-context example construction and for prediction from local neighborhoods [46], [47], [48]. In program synthesis, CYANEA selects among symbolic solvers and LLM-prompt pairs with rewards that may include success, time, and token cost [11]. Budgeted inference further shows that allocating computation across samples or reasoning trajectories can improve performance under fixed resources [49], [50]. Censored-regression and survival-analysis methods address outcomes observed only as bounds, such as costs known only to exceed a cap [51], [52], [53]. Prior work therefore contributes adaptive selection, local evidence transfer, or cost modeling, yet none directly addresses sequential multi-workflow code generation.

III. GENERATION WORKFLOWS

We instantiate three workflows that span distinct inference mechanisms and costs, from a single pass to added planning structure and post-hoc repair. Direct Coding is the standard single-pass setting in code LLM evaluation [1], [25], [24], [27] and serves as our baseline. Plan-Structured Coding draws on planning-based prompting, which improves multi-step reasoning and code structure [15], [54], [55], [56], [57], and Test-Guided Repair adopts repair-oriented workflows, shown to improve reliability under functional evaluation [17], [37], [18], [58], [19], [38]. We design the latter two to be budget-conscious, folding the benefits of planning and repair into as few inference calls as possible, since under a shared task budget every extra call competes with later fallbacks.¹

a) *Direct Coding*: Given a task, the model emits the final solution in one call, preserving the required signature, scaffold, library constraints, and behavioral contract [23]. It is the cheapest workflow and produces no intermediate artifact, but folds interpretation, interface matching, and implementation into a single response, leaving no structure to catch early misunderstandings.

b) *Plan-Structured Coding*: The model drafts a concise implementation checklist and then produces the final solution in the same response. The checklist guides generation as local context rather than a separate planning request. It fixes the interface, side effects, return contract, and failure contract, locking the scaffold, names, imports, and I/O format

¹Full prompts and implementation details for each workflow are provided in the README of our replication package.

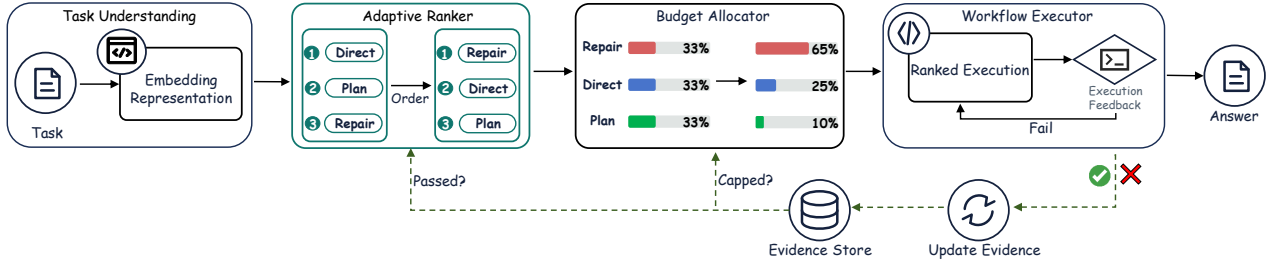


Fig. 1. Overview of BARA, performing ranked workflow execution with task-level budget allocation.

while excluding validation or recovery logic not stated by the task. Only the final code is executed. Unlike plan-then-code pipelines that spend one call on planning and another on implementation [15], [54], [55], the structure lives inside a single generation, adding modest completion cost but no extra request.

c) Test-Guided Repair: In one response the model produces a draft implementation together with a small task-specific self-test suite, merging the program- and test-generation steps that prior pipelines run as separate calls [17], [37], [18], [58]. The system runs the tests locally; if the draft passes, it is evaluated directly; otherwise the model sees the draft, the tests, and the error log and returns one repaired implementation. Two choices make this budget-conscious: joint draft-and-test generation removes a dedicated test call while still surfacing boundary cases before repair, and the repair call is conditional, drawing on the task budget only when the draft fails. The workflow builds on self-debugging and self-refinement methods [18], [38], [59] but differs by generating self-tests jointly with the draft and repairing only conditionally under the shared budget.

IV. PROBLEM FORMULATION

We model code generation under a fixed token budget as an episodic Markov decision process: for one task, a policy repeatedly selects which workflow to attempt next and how much budget to grant it.

a) Setup: Let x denote a code generation task and $\mathcal{G}$ the pool of candidate inference workflows, with $M = |\mathcal{G}|$; here $\mathcal{G}$ contains the three workflows of Section III. We treat each $g \in \mathcal{G}$ as a single budgeted action, attempted at most once per task: when run to completion on x , it consumes an accounted-token cost $C_g(x) > 0$ (the sum over its internal model calls) and yields a candidate with acceptance outcome $S_g(x) \in \{0, 1\}$, the pass/fail signal of the benchmark evaluator. The joint law of $(C_g(x), S_g(x))$ is unknown and varies with the task. Restricting each workflow to one attempt keeps the focus on coordinating heterogeneous workflows rather than allocating repeated draws to one; repeated sampling is therefore a baseline, not part of the action space.

b) Episode: Each task forms one episode with budget B . The state before attempt j is (A_j, R_j) , the set $A_j \subseteq \mathcal{G}$ of attempted workflows and the remaining budget R_j , with $A_1 = \emptyset$ and $R_1 = B$. An action is a pair (g_j, b_j) : an untried

workflow $g_j \in \mathcal{G} \setminus A_j$ and a cap $b_j \in (0, R_j]$ under which g_j runs, truncated once its accounted cost reaches b_j . The attempt *succeeds* if $C_{g_j}(x) \leq b_j$ and $S_{g_j}(x) = 1$; otherwise it *fails*, either by rejection (the workflow completes, charging the exact $C_{g_j}(x)$) or by truncation (charging b_j , with the true cost known only to exceed it). Either way the charged cost is $\min(C_{g_j}(x), b_j)$, giving $R_{j+1} = R_j - \min(C_{g_j}(x), b_j)$ and $A_{j+1} = A_j \cup \{g_j\}$. The episode ends at the first success, or once the pool is exhausted ($A_{j+1} = \mathcal{G}$) or the budget is spent ($R_{j+1} = 0$); the horizon is at most M . The two failure modes leave different evidence: rejection reveals the exact cost, whereas truncation right-censors it at the cap. The cost model of Section V relies on this distinction.

c) Objective: A policy π maps each state to an action, inducing on every task an execution order and a per-attempt cap: exactly the ranked workflow execution with task-level budget allocation studied here. The primary objective is the solve probability under the budget,

$$\max_{\pi} \Pr_{\pi}[x \text{ is solved within } B]; \quad (1)$$

among policies with comparable solve probability, we prefer lower expected charged cost, the secondary criterion targeted by the token-aware variant of Section V.

d) Structural properties: The value of an attempt combines an *immediate term*, the probability that it completes and is accepted, and a *continuation value*, the expected return of the budget it leaves to later workflows; their coupling motivates the heuristic of Section V. *Last attempt:* for the final reachable workflow no later workflow remains, the continuation value vanishes, and success is nondecreasing in the cap, hence granting it all remaining budget is weakly optimal. *Coupling:* for earlier attempts a larger cap raises both the success probability and the charged cost $\min(C_g, b)$, draining budget from later attempts on failure, hence ranking and allocation are coupled and an expensive early failure can put a cheaper complementary workflow out of reach.

V. THE BARA POLICY

BARA is a policy for the budgeted execution problem of Section IV. It factors the action (g_j, b_j) into three components, shown in Fig. 1: an adaptive ranker that orders the pool by estimated utility, a budget allocator that assigns each attempt a chance-constrained token cap, and a workflow executor that walks the order, stops at the first acceptance, and writes every

outcome back to the trace store. All quantities are estimated at decision time from executions retrieved from similar tasks; the policy has no trained parameters.

A. Evidence Retrieval

The policy maintains a trace store $\mathcal{D} = \{(x_i, g_i, b_i, \delta_i, \tilde{c}_i, s_i)\}$ holding every executed attempt, where $\delta_i \in \{0, 1\}$ flags completion, $\tilde{c}_i$ is the charged cost, and s_i the acceptance outcome; when $\delta_i = 0$ the completion cost is right-censored at the cap b_i . Each task is represented by an embedding of its visible problem description, the information available before any workflow runs, and task similarity is the cosine distance between normalized embeddings. For a target task x , the policy retrieves the set $N_k(x)$ of the k most similar previously executed tasks represented in $\mathcal{D}$; the encoder and k are fixed across all experiments. Because each episode executes only a prefix of the pool, a neighboring task contributes evidence for workflow g only if g was actually executed on it,

$$I_g(x) = \{i : x_i \in N_k(x) \text{ and } g \text{ was executed on } x_i\}, \quad (2)$$

where x_i denotes the i -th task recorded in $\mathcal{D}$.

B. Utility Estimation and Workflow Ranking

Every executed attempt is stored with a scalar utility $u \in [0, 1]$. We consider two designs,

$$u^{\text{PO}} = s, \quad u^{\text{TA}} = s \left(1 - \left(\frac{\tilde{c}}{B}\right)^2\right), \quad (3)$$

where $s \in \{0, 1\}$ is the acceptance outcome of the attempt, $\tilde{c}$ its charged cost, and B the budget of its task. The pass-only design u^{PO} records acceptance alone. The token-aware design u^{TA} keeps the same acceptance requirement but discounts expensive successes; it internalizes the continuation effect identified in Section IV, since a success that consumes most of the budget would leave little room for fallbacks on a similar future task and is therefore weaker evidence for an early slot than an equally correct but cheaper one. Failed and truncated attempts have $s = 0$ and score zero under both designs, and cost never earns utility without acceptance. An accepted attempt completes within its cap, which never exceeds the task budget, hence both utilities lie in $[0, 1]$. The utility definition controls which historical executions count as strong evidence, while the ranking rule below is unchanged.

The ranker estimates the utility of workflow g on task x by the local empirical mean over the retrieved evidence,

$$\hat{\mu}_g(x) = \begin{cases} \frac{1}{|I_g(x)|} \sum_{i \in I_g(x)} u_{i,g}, & |I_g(x)| > 0, \\ 0, & |I_g(x)| = 0, \end{cases} \quad (4)$$

where $u_{i,g}$ is the utility recorded when g was executed on x_i . The ranked order $\sigma(x)$ sorts the pool by descending $\hat{\mu}_g(x)$, breaking ties uniformly at random. Relative to the formulation of Section IV, this rule greedily maximizes the immediate term and leaves the continuation value to the cap and the token-aware utility.

A workflow accrues evidence only when it runs, letting one that is rarely reached under a tight budget keep a stale estimate once another workflow pulls ahead. Evidence-based workflow ranking therefore requires an initial coverage phase to make the local utility estimates defined for all workflows and avoid evidence imbalance at the start of the online stream. In this phase, the least-executed workflow is placed first until every workflow has at least n_0 executions [60]. Because tied workflows are randomized rather than dropped and the executor walks the full order under the task budget, a lower-ranked workflow is still reached whenever earlier attempts fail and budget remains. As evidence accumulates beyond the coverage phase, the empirical estimates dominate and the ranking becomes increasingly exploitative.

C. Cost Model and Chance-Constrained Allocation

The cap mediates the tension identified in Section IV: it must be large enough that the current attempt completes, yet tight enough that a failure leaves budget for fallbacks. The allocator resolves this with a chance constraint. Each attempt is funded with the smallest cap under which it completes with high probability under a local cost model, and the remainder is left to fallbacks.

Completion costs are estimated from the neighborhood executions of the same workflow. For workflow g , the allocator retrieves the set $N_{g,k}(x)$ of the k nearest historical executions of g to x from $\mathcal{D}$. Completed attempts reveal exact costs, while truncated attempts reveal only that the cost exceeded the assigned cap, entering as right-censored observations. We fit a log-normal cost law locally, with parameters estimated from $N_{g,k}(x)$ rather than shared globally,

$$(\hat{m}_g, \hat{s}_g) = \arg \max_{m, s > 0} \left[\sum_{i: \delta_i = 1} \log f(\tilde{c}_i; m, s) + \sum_{i: \delta_i = 0} \log(1 - F(b_i; m, s)) \right], \quad (5)$$

where the sums range over the observations in $N_{g,k}(x)$, and f and F are the log-normal density and distribution function. The density term uses the exact costs of completed attempts, and the survival term accounts for truncated attempts whose completion costs are unobserved.

The fitted law is converted into a cap through its upper tail,

$$q_g(x) = \hat{F}_g^{-1}(1 - \epsilon) = \exp(\hat{m}_g + \Phi^{-1}(1 - \epsilon) \hat{s}_g), \quad (6)$$

where $\hat{F}_g$ is the fitted cost distribution and Φ^{-1} the inverse standard normal distribution function. By construction, $q_g(x)$ is the smallest cap under which the attempt completes with probability at least $1 - \epsilon$ under the fitted model, so the rule targets truncation risk directly rather than average cost. We set $\epsilon = 0.01$. The assigned cap is clipped by the remaining budget,

$$b_j = \min(q_{g_j}(x), R_j). \quad (7)$$

If g_j is the last workflow in the order, it instead receives all remaining budget, $b_j = R_j$, matching the optimality property derived in Section IV. When the neighborhood does not provide enough usable observations to fit Eq. (5) stably, the allocator falls back to splitting the remaining budget across the not-yet-attempted workflows in proportion to fixed prior weights, set to 1:1:2 for Direct Coding, Plan-Structured Coding, and Test-Guided Repair.

D. Ranked Execution

The executor attempts the workflows in the order $\sigma(x)$. Before each attempt the allocator assigns a cap from the remaining budget; the attempt then runs under that cap and is checked against the acceptance signal. On acceptance, execution stops and the task is solved. On failure or truncation, the charged cost is deducted and the executor proceeds to the next workflow while budget remains. Every attempt is appended to $\mathcal{D}$, letting truncated attempts enter future cost fits as censored evidence rather than being discarded. Ranking and allocation are decoupled: the ranker fixes which workflow runs next, and the allocator fixes how much it may spend. This factorization is itself part of the greedy design, since caps are chosen per attempt given the order rather than jointly with it.

VI. STUDY DESIGN

This study evaluates BARA’s orchestration of code-generation workflows under task-level inference budgets. To isolate the contribution of orchestration, we hold the benchmarks, models, workflow pool, task representation, reward definitions, and budget protocol fixed across all compared methods, varying only how workflows are sampled or ranked, how budget is divided across attempts, and how execution feedback is used. We organize the evaluation around four research questions:

RQ1: Does workflow coordination improve correctness over single-workflow baselines? RQ1 examines whether the workflow pool offers usable complementarity beyond the strongest individual workflow. Contrasting BARA with single-workflow execution and repeated sampling isolates the gains of task-specific workflow coordination from those attainable by a single fixed workflow under one-shot or repeated invocation.

RQ2: Does BARA place successful workflows earlier in the execution order? RQ2 asks whether BARA assigns higher execution priority to workflows likely to solve the current task. Under a finite task budget, an otherwise successful workflow may fail to contribute if earlier attempts consume the budget required for its execution; ordering therefore determines which workflows remain reachable before termination.

RQ3: Does BARA’s adaptive allocation improve budgeted workflow performance? RQ3 examines how allocation strategy affects performance under a shared task-level budget. Allocation determines whether the current workflow receives enough budget to run to completion rather than being truncated at its cap; we compare adaptive caps against proportional, equal, and empirical-quantile alternatives.

RQ4: How does token-aware feedback affect the cost-accuracy tradeoff? RQ4 studies whether feedback should account for the token cost of successful attempts. By discounting budget-heavy successes, token-aware feedback may shift early rankings toward lower-cost solving workflows; we compare it with pass-only feedback on solve rate and mean cost.

A. Models

We evaluate three instruction-tuned LLMs for code generation: *Qwen2.5-Coder-32B-Instruct* [61], *Qwen3-Coder-30B-A3B-Instruct* [62], and *Claude Opus-4.8*², referred to as Qwen2.5-32B, Qwen3-30B, and Opus-4.8 hereafter. Qwen2.5-32B is a dense code-specialized transformer, Qwen3-30B uses a mixture-of-experts architecture, and Opus-4.8 is a closed-weight general-purpose frontier model. This diversity in architecture, specialization, and openness allows us to evaluate whether BARA generalizes across models with different designs.

B. Task Representation and Similarity

We instantiate the task representation and neighborhood of Section V as follows. Each task’s visible problem description is embedded with Qwen3-Embedding-0.6B [63], producing a 1024-dimensional vector, and task similarity is measured by cosine distance between normalized embeddings. For every target task, the system retrieves the $k = 30$ most similar previously evaluated tasks in the main experiments. For the initial coverage phase, we set $n_0 = 20$; with three workflows, this gives at least 60 initial workflow executions, which centers the default $k = 30$ within the neighborhood-size sweep of the sensitivity analysis.

C. Budget Regimes and Allocation

Budgets are measured in accounted tokens instead of raw token counts to reflect the asymmetric pricing of prompt and generated tokens in commercial LLM APIs. We assign unit weight to prompt tokens and weight four to generated tokens, defining the accounted cost as $c = n_{\text{prompt}} + 4n_{\text{completion}}$. The 1:4 weighting corresponds to the modal input-to-output pricing ratio reported across major commercial LLM APIs and is consistent with recent cost analyses of LLM-based code generation and inference [64], [65], [66]. Under this accounting scheme, longer generations consume a proportionally larger share of the task budget and reduce the budget available to subsequent workflows.

We evaluate three per-task budget regimes: 10k, 15k, and 20k accounted tokens. These values are coarse budget levels rather than task-specific tuned thresholds. The 10k regime represents a *constrained* setting in which expensive workflows, especially repair-oriented attempts, are frequently capped. The 15k regime is an *intermediate* setting where many attempts can complete, but long generations and repair-heavy cases still compete with fallback workflows for budget. The 20k regime is a *loose* setting in which most individual attempts fit and nearly all fallbacks can run. Coverage therefore saturates,

²<https://www.anthropic.com/claude/opus>

TABLE I
CORRECTNESS AND COST OF WORKFLOW ORCHESTRATION ACROSS BUDGET REGIMES.

Model	Method	Solve rate			Reward			Cost (k/task)			Δ Best		
		10k	15k	20k	10k	15k	20k	10k	15k	20k	10k	15k	20k
Qwen3-30B	Direct Coding	47.5%	47.5%	47.5%	0.475	0.475	0.475	1.10	1.10	1.10	—	—	—
	Plan-Structured Coding	47.7%	47.7%	47.7%	0.477	0.477	0.477	1.95	1.95	1.95	—	—	—
	Test-Guided Repair	53.6%	54.9%	55.0%	0.536	0.549	0.550	5.89	5.93	5.93	—	—	—
	Repeated Sampling (Direct Coding)	54.3%	55.6%	55.9%	0.543	0.556	0.559	5.35	7.59	9.80	▲ 0.7%	▲ 0.7%	▲ 0.9%
	Repeated Sampling (Plan-Structured)	53.9%	55.7%	56.6%	0.539	0.557	0.566	5.82	8.08	10.24	▲ 0.3%	▲ 0.8%	▲ 1.6%
	Repeated Sampling (Test-Guided)	54.6%	57.7%	59.4%	0.546	0.577	0.594	7.65	9.82	11.79	▲ 1.0%	▲ 2.8%	▲ 4.4%
	★BARA	59.0%*	61.9%*	62.1%*	0.590	0.619	0.621	5.23	6.95	7.04	▲ 5.4%	▲ 7.0%	▲ 7.1%
	👑Budgeted Oracle	61.8%	62.1%	62.2%	0.618	0.621	0.622	—	—	—	—	—	—
Qwen2.5-32B	Direct Coding	48.7%	48.7%	48.7%	0.487	0.487	0.487	1.07	1.07	1.07	—	—	—
	Plan-Structured Coding	49.0%	49.0%	49.0%	0.490	0.490	0.490	2.16	2.16	2.16	—	—	—
	Test-Guided Repair	51.8%	54.6%	54.7%	0.518	0.546	0.547	6.13	6.23	6.24	—	—	—
	Repeated Sampling (Direct Coding)	56.4%	57.7%	58.3%	0.564	0.577	0.583	5.18	7.18	8.74	▲ 4.6%	▲ 3.1%	▲ 3.6%
	Repeated Sampling (Plan-Structured)	55.4%	56.8%	57.5%	0.554	0.568	0.575	5.67	7.87	9.97	▲ 3.6%	▲ 2.2%	▲ 2.8%
	Repeated Sampling (Test-Guided)	51.5%	58.8%	61.3%	0.515	0.588	0.613	7.82	10.04	11.94	▼ 0.3%	▲ 4.2%	▲ 6.6%
	★BARA	58.0%*	61.1%*	61.4%*	0.580	0.611	0.614	4.95	6.55	7.14	▲ 6.2%	▲ 6.5%	▲ 6.7%
	👑Budgeted Oracle	60.9%	61.5%	61.6%	0.609	0.615	0.616	—	—	—	—	—	—
Opus-4.8	Direct Coding	66.5%	66.5%	66.5%	0.665	0.665	0.665	1.25	1.25	1.25	—	—	—
	Plan-Structured Coding	67.1%	67.1%	67.1%	0.671	0.671	0.671	3.05	3.05	3.05	—	—	—
	Test-Guided Repair	63.3%	70.7%	70.9%	0.633	0.707	0.709	7.46	7.64	7.64	—	—	—
	Repeated Sampling (Direct Coding)	73.5%	73.9%	74.1%	0.735	0.739	0.741	3.80	5.11	6.41	▲ 6.4%	▲ 3.2%	▲ 3.2%
	Repeated Sampling (Plan-Structured)	70.5%	73.9%	74.8%	0.705	0.739	0.748	5.20	6.60	7.88	▲ 3.4%	▲ 3.2%	▲ 3.9%
	Repeated Sampling (Test-Guided)	63.6%	71.9%	74.5%	0.636	0.719	0.745	8.15	9.68	11.00	▼ 3.5%	▲ 1.2%	▲ 3.6%
	★BARA	72.4%	76.3%*	76.9%*	0.724	0.763	0.769	3.97	5.37	7.07	▲ 5.3%	▲ 5.6%	▲ 6.0%
	👑Budgeted Oracle	76.5%	77.0%	77.0%	0.765	0.770	0.770	—	—	—	—	—	—

and the ranking matters mainly for solving a task on the first attempt rather than after fallbacks. These round levels avoid selecting highly specific cutoffs around observed cost distributions while still covering scarce, moderate, and largely sufficient budget conditions.

D. Baselines and Metrics

Solve rate, our primary correctness metric, is the fraction of tasks for which an accepted solution is found within the task budget. First-hit rate is the fraction of tasks solved by the first attempted workflow. Accounted-token cost is the mean per-task cost, reported in thousands. Capped-attempt rate is the fraction of attempted workflows that reach their assigned cap. Reward follows the pass-only and token-aware definitions of Section V-B.

Single-workflow baselines run one workflow once per task; repeated sampling re-executes the same workflow until success or budget exhaustion; static ranking baseline evaluates all $3! = 6$ permutations of the workflow pool; uniform ranking baseline samples an order uniformly per task; and budgeted oracle reports the maximum solve rate attainable by selecting any successful workflow under the same budget. The allocation baselines are proportional allocation, which uses a static 1:1:2 split that gives Test-Guided Repair twice the share of the two coding workflows, since it issues two inference passes rather than one; equal allocation, which is cost-blind and splits the budget evenly; and empirical-quantile allocation, which caps each workflow at the 99th percentile of neighbor cost but estimates it from completed neighbors only, discarding censored attempts. Matching the BARA allocator’s percentile isolates the effect of dropping censored observations.

We report BARA as the mean over the canonical benchmark order and ten uniform task-order permutations using seeds 0 through 9. The non-adaptive rankings are reported at their

strongest realization by solve rate, including their first-hit rate: the static ranking baseline is the best of its six permutations, and the uniform ranking baseline is the best of its ten runs over the same seeds 0 through 9. We assess statistical significance with a paired bootstrap that resamples tasks with replacement and resamples the task-order streams as whole clusters; over 10,000 iterations we recompute the mean paired difference and mark it significant (*) when the 95% interval excludes zero.

A star marks a significant paired gain for BARA on the starred metric: in Tables I and V over the strongest repeated-sampling baseline, with the separate gain over the best single workflow significant in all settings; in Table II over the static or uniform ranking with the higher solve rate, against which both solve rate and first-hit are evaluated; in Table III over the best non-adaptive allocation policy; and in Table IV over pass-only feedback on the token-aware cost reduction.

E. Experimental Environment

All experiments are conducted on a system running Ubuntu 24.04.3 LTS with GNU/Linux kernel version 6.8.0-85-generic. The hardware configuration consists of four NVIDIA L40S GPUs, each equipped with 48 GB of VRAM. Open-weight model inference is performed using vLLM [67], while Opus-4.8 inference uses the Anthropic Message Batches API.

VII. EVALUATION

We evaluate BARA under fixed task-level budgets, measuring how coordinating workflows affects correctness and mean per-task inference cost.

A. RQ1: Does workflow coordination improve correctness over single-workflow baselines?

Table I shows that BARA outperforms the best individual workflow in all nine model-budget pairs, by +5.3% to +7.1%,

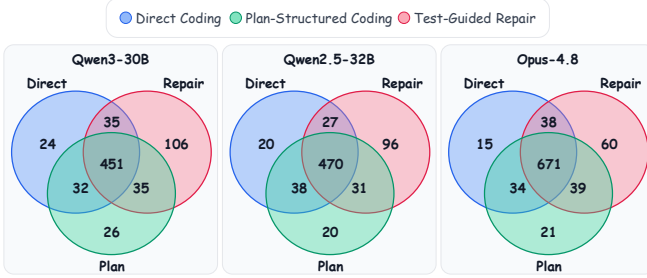


Fig. 2. Solved-task overlap among the three workflows.

and the gain is statistically significant in every one. The margin widens as the budget loosens: on Qwen3-30B it grows from +5.4% at 10k to +7.1% at 20k, because a looser budget admits the complementary workflows that tight caps would truncate. Under pass-only feedback, the reward column equals solve rate, meaning that the reported gains correspond directly to additional solved tasks.

Repeated sampling is the stronger same-budget baseline, spending the same budget on additional attempts from one workflow. BARA outperforms it in eight of the nine pairs, significantly in seven, while the two remaining cases delimit where repeated sampling stays competitive. On Opus-4.8 in the constrained regime it trails repeated Direct Coding, 72.4% against 73.5%. Opus already reaches 66.5% with Direct Coding alone at 1.25K per task; under the tightest budget, pure coverage from cheap direct samples outpaces a multi-workflow schedule. This reverses as the budget loosens, with BARA overtaking repeated sampling at 15k and 20k. On Qwen2.5-32B in the loose regime the gain over repeated Test-Guided Repair, 61.4% against 61.3%, is not significant, yet BARA reaches the same coverage at 7.14K rather than 11.94K accounted tokens per task, a correctness tie that still yields a substantial cost reduction.

The cost column shows the gain comes from selective use of expensive workflows, not from always running them. Test-Guided Repair is the strongest single workflow on the Qwen models but also the most expensive. In the constrained regime BARA achieves a higher solve rate at lower cost, reaching 58.0% at 4.95K where repair reaches 51.8% at 6.13K on Qwen2.5-32B, and 59.0% at 5.23K where repair reaches 53.6% at 5.89K on Qwen3-30B. On Opus-4.8, where Plan-Structured Coding is the cheapest strong workflow (67.1% at 3.05K), BARA reaches 72.4% at 3.97K by adding costlier workflows only when they help. Orchestration therefore decides when an expensive workflow is worth attempting under a shared budget.

The advantage is not merely cap avoidance; it persists, and even grows, as the budget loosens. In the loose regime BARA matches or outperforms the strongest repeated-sampling baseline on all three models and lands within 0.2% of the budgeted oracle in every case, while costing far less. BARA nears the oracle because a loose budget lets it walk the whole pool and reach the same workflow union the oracle measures. Repeated

TABLE II
SOLVE RATE AND FIRST-HIT RATE BY RANKING STRATEGY. STATIC AND UNIFORM ROWS REPORT THE STRONGEST REALIZATION BY SOLVE RATE.

Model	Ranking	Solve rate			First-hit rate		
		10k	15k	20k	10k	15k	20k
Qwen3-30B	● Static	56.3%	60.5%	62.1%	47.1%	49.0%	49.4%
	● Uniform	55.8%	60.5%	62.1%	46.3%	49.6%	50.1%
	★ BARA	59.0%*	61.9%*	62.1%	48.4%	53.6%*	54.1%*
Qwen2.5-32B	● Static	54.5%	60.2%	61.1%	46.4%	49.6%	50.4%
	● Uniform	54.4%	60.1%	61.1%	46.1%	48.3%	49.2%
	★ BARA	58.0%*	61.1%*	61.4%	48.1%*	52.2%*	53.3%*
Opus-4.8	● Static	69.4%	74.4%	76.1%	62.1%	66.1%	65.6%
	● Uniform	68.7%	75.1%	76.1%	61.8%	65.8%	66.1%
	★ BARA	72.4%*	76.3%*	76.9%	64.2%*	66.4%	68.0%*

sampling’s cost scales with the budget ceiling rather than task difficulty, since it keeps drawing samples until a solution is accepted or the budget is exhausted: on Qwen3-30B the per-task cost of repeated Direct Coding rises from 5.35K at the 10k budget to 9.80K at the 20k budget, whereas BARA stops at the first success and grows only from 5.23K to 7.04K, roughly 40% below the strongest repeated-sampling baseline at 11.79K. Matching the oracle while undercutting repeated sampling on cost is the clearest sign that the gains come from coordinating complementary workflows, not from spending more attempts on one.

Figure 2 locates this headroom. On the Qwen models most solved tasks fall in the three-way intersection, 451 for Qwen3-30B and 470 for Qwen2.5-32B, where workflow choice barely matters; the gains instead come from the 258 and 232 tasks outside it, where success is workflow-dependent. Opus-4.8 has a larger intersection of 671 tasks among the 878 solved by any workflow, yet still leaves 207 in pairwise and workflow-specific regions, enough complementarity for BARA to beat repeated sampling in the intermediate and loose regimes though repeated Direct Coding wins in the constrained one.

Answer to RQ1: BARA significantly outperforms every individual workflow in all nine settings and the stronger repeated-sampling baseline in seven, usually at lower cost. The gain comes from complementary coverage: different workflows solve different task subsets, and BARA converts that complementarity into a higher solve rate under a fixed budget.

B. RQ2: Does BARA place successful workflows earlier in the execution order?

Solve rate and first-hit rate separate along the budget axis (Table II). BARA significantly improves solve rate at the constrained and intermediate budgets on all three models, but the gap closes to non-significance at 20k. Once the budget is loose enough to exhaust the pool, every ranking reaches the same solving workflow, driving solve rate to the budgeted-oracle ceiling no matter how the pool is ranked. First-hit is more durable: because the ranking fixes the first attempt regardless of remaining budget, BARA has the highest first-hit rate in every cell, significant in seven of the nine settings and, unlike solve rate, still significant in the loose regime on all three models. Ranking quality therefore surfaces in solve rate while the budget is tight enough for the first choice to matter, and in first-hit once the budget loosens and fallbacks erase the solve-rate gap.

TABLE III
SOLVE RATE AND CAPPED RATE BY ALLOCATION STRATEGY.

Model	Allocation	Solve rate			Capped rate		
		10k	15k	20k	10k	15k	20k
Qwen3-30B	● Proportional	56.1%	60.6%	61.8%	15.6%	5.7%	1.5%
	● Equal	56.5%	59.6%	61.2%	13.7%	6.9%	3.0%
	● Empirical Quantile	57.5%	60.8%	61.2%	17.8%	6.0%	3.7%
	★ BARA	59.0%*	61.9%*	62.1%*	9.7%*	1.2%*	0.2%*
Qwen2.5-32B	● Proportional	56.9%	60.2%	60.9%	12.6%	3.9%	2.8%
	● Equal	56.5%	58.9%	60.4%	12.7%	6.3%	2.7%
	● Empirical Quantile	55.9%	61.0%	61.1%	17.7%	3.9%	3.5%
	★ BARA	58.0%*	61.1%*	61.4%*	10.3%*	1.6%*	0.8%*
Opus-4.8	● Proportional	67.9%	74.7%	75.4%	30.5%	6.4%	6.4%
	● Equal	71.0%	74.0%	75.4%	18.5%	7.5%	3.0%
	● Empirical Quantile	66.1%	73.0%	75.8%	33.0%	12.2%	5.8%
	★ BARA	72.4%*	76.3%*	76.9%*	16.2%*	3.6%*	1.2%*

On Qwen3-30B, the solve-rate gain is a significant +2.7% at 10k and +1.4% at 15k, then narrows to a three-way tie of 62.1% at 20k, while first-hit shows the mirror pattern: a non-significant +1.3% at the tightest budget becomes a significant +4.0% at both 15k and 20k. Qwen2.5-32B has the same solve-rate shape, significant at 10k and 15k and non-significant at 20k, but its first-hit lead is significant at every budget, growing from +1.7% to +2.9% as the budget loosens. Opus-4.8 is the only model whose first-hit lead is non-monotonic, significant at +2.1% at 10k, non-significant at +0.3% at 15k, then significant again at +1.9% at 20k, while its solve-rate gain follows the common pattern, significant at 10k and 15k and non-significant at 20k. Across models, a first-hit lead becomes a solve-rate lead only while the first choice still changes final coverage; once fallbacks recover the same tasks, solve rate converges to the ceiling and first-hit is the remaining signal that separates the rankings.

Answer to RQ2: BARA improves ranking efficiency. It significantly raises solve rate under constrained and intermediate budgets, where the first attempt still decides which workflows stay reachable, and that gap closes to non-significance once the budget is loose, as any ranking that exhausts the pool reaches the same solving workflow. The first-hit advantage is more durable, holding in every cell and remaining the signal that separates the rankings once the budget loosens.

C. RQ3: Does BARA’s adaptive allocation improve budgeted workflow performance?

The BARA allocator attains the best solve rate in all nine settings and significantly lowers the capped-attempt rate in every one (Table III). Its solve-rate margins peak in the constrained regime, reaching +2.9% over proportional and +2.5% over equal allocation on Qwen3-30B, and +4.5% over proportional allocation on Opus-4.8.

Equal allocation is the natural cost-blind default, dividing the budget without any model of per-workflow cost, which makes it the standard reference point. The capped-rate column shows why this fails: it still caps 3.0% of attempts on Qwen3-30B in the loose regime and 7.5% on Opus-4.8 in the intermediate one, against 0.2% and 3.6% under the BARA allocator. Dividing the budget evenly ignores that the workflows differ several-fold in cost, starving the expensive workflow while the cheap ones waste their share.

Proportional allocation is the cost-aware counterpart, reserving budget by a fixed ratio that anticipates each workflow’s relative cost rather than ignoring it. This helps on the Qwen models, where repair genuinely warrants a larger share, but a ratio fixed in advance cannot adapt to cost that varies by task and model. On Opus-4.8 in the constrained regime, the same reservation backfires: it over-commits to repair, caps 30.5% of attempts, and reaches only 67.9%, below even equal allocation at 71.0%, whereas the BARA allocator caps 16.2% and reaches 72.4%. The contrast with equal allocation is instructive: a fixed cost ratio helps only when it matches the task, and a mismatched ratio does worse than ignoring cost altogether. Even where the split is better matched, as on Opus-4.8 in the loose regime, proportional still caps 6.4% of attempts against 1.2%, and the learned cap mainly avoids truncating workflows that would otherwise complete.

Empirical-quantile allocation is the closest competitor because it also conditions on local cost, but dropping censored observations discards exactly the cases that signal a longer required cost. A truncated attempt reveals only that its true cost exceeded the cap; discarding it biases the fitted cost downward and yields caps that are systematically too low. In the constrained regime it caps far more attempts than the BARA allocator, 17.8%, 17.7%, and 33.0% against 9.7%, 10.3%, and 16.2%, and trails on solve rate by 1.5 to 6.3%; the gap narrows as the budget loosens and censoring becomes rare, but the higher capped rate persists. Modeling the censored attempts rather than discarding them translates into a measurable solve-rate gain at the matched percentile.

Answer to RQ3: The BARA allocator raises correctness on top of ranking by sizing each cap from a local cost model that includes the censored costs of capped attempts. Proportional and equal allocation apply caps fixed in advance that miss task-specific cost. Empirical-quantile allocation conditions on cost but discards censored attempts, biasing its caps downward and truncating workflows prematurely.

TABLE IV
BARA UNDER PASS-ONLY (PO) AND TOKEN-AWARE (TA) FEEDBACK.

Model	Budget	Solve rate		Cost (K/task)		Reward	
		PO	TA	PO	TA	PO	TA
Qwen3-30B	10k	59.0%	58.9%	5.23	4.85*	0.590	0.523
	15k	61.9%	61.8%	6.95	5.59*	0.619	0.560
	20k	62.1%	62.0%	7.04	5.99*	0.621	0.580
Qwen2.5-32B	10k	58.0%	58.3%	4.95	4.77*	0.580	0.529
	15k	61.1%	61.3%	6.55	5.40*	0.611	0.561
	20k	61.4%	61.4%	7.14	5.73*	0.614	0.578
Opus-4.8	10k	72.4%	72.5%	3.97	3.91*	0.724	0.682
	15k	76.3%	76.5%	5.37	4.51*	0.763	0.719
	20k	76.9%	77.0%	7.07	4.62*	0.769	0.741

D. RQ4: How does token-aware feedback affect the cost-accuracy tradeoff?

Compared with BARA-PO in Table IV, BARA-TA keeps solve rate within 0.3% in every setting, with no statistical significance in solve-rate changes, so coverage is preserved. Meanwhile the mean per-task cost falls in all nine settings, the

TABLE V
GENERALIZATION TO LIVECODEBENCH ACROSS BUDGET REGIMES.

Model	Method	Solve rate			Cost (K/task)		
		10k	15k	20k	10k	15k	20k
Qwen3-30B	Direct	51.9%	52.8%	53.0%	4.40	5.15	5.38
	Plan	51.0%	51.8%	51.9%	4.52	5.29	5.58
	Repair	49.0%	50.1%	50.8%	5.72	6.75	7.49
	Repeated (Plan)	53.0%	54.7%	55.6%	6.22	8.55	10.80
	★ BARA	53.9%	57.5%*	59.0%*	6.34	8.08	9.56
	👑 Oracle	59.6%	60.8%	60.9%	—	—	—
Qwen2.5-32B	Direct	42.0%	42.0%	42.0%	3.89	4.36	4.57
	Plan	42.4%	42.4%	42.4%	2.68	2.83	2.90
	Repair	40.1%	40.3%	40.7%	4.84	5.32	5.69
	Repeated (Plan)	45.6%	49.0%	49.8%	6.48	8.96	10.19
	★ BARA	49.0%*	51.2%*	51.6%*	6.49	7.86	8.65
	👑 Oracle	52.3%	52.3%	52.3%	—	—	—

reduction being statistically significant in each. The saving is small in the constrained regime, which already has little slack, and grows as the budget loosens, reaching 2.45K accounted tokens per task on Opus-4.8 at 20k and 1.0 to 1.4K on the Qwen models at the looser budgets.

This reflects a change in the ranking signal, not a discount applied to the output. A cheaper success counts as stronger evidence than an equally correct but budget-heavy one, and BARA-TA moves workflows that tend to succeed cheaply earlier while leaving costlier workflows as fallback. The reward columns make this explicit. Each policy is scored under its own utility, and BARA-TA’s lower reward follows by construction, as the token-aware utility discounts the expensive successes that pass-only feedback counts in full. Solve rate, not reward, is the comparable coverage metric across the two policies. Passing attempts then consume fewer tokens, lowering mean cost without giving up solved tasks. The reduction is largest on Opus-4.8 at the looser budgets, where pass-only feedback leaves the most expensive successful attempts ranked early and the token-aware ranking has the most room to reach a working solution through a cheaper workflow first. Because solve rate is unchanged and cost only decreases, token-aware feedback is a one-sided improvement on the cost axis under every budget we test, not a tradeoff that sacrifices coverage for cost.

Answer to RQ4: BARA-TA improves the cost–accuracy tradeoff by reshaping the ranking evidence to count a budget-compatible success as stronger than an equally correct but expensive one. It lowers mean per-task cost, significant in all nine settings, while holding solve rate within 0.3%.

VIII. EXTENDED ANALYSIS

A. Generalization Analysis

To assess the generalization capability of BARA, we evaluate it on LiveCodeBench, drawn from competitive-programming problems, reporting the two open-weight Qwen models in Table V. BARA attains the highest solve rate on both models across all three budgets, significantly exceeding the best single workflow by up to 9.2% on Qwen2.5-32B and 6.0% on Qwen3-30B. The three workflows lie within roughly 2% of one another on each model, a far narrower spread than any single one’s gap to BARA, indicating that the gain arises

from coordination across them, since no single workflow dominates this distribution. Among repeated-sampling baselines we report the strongest, repeated sampling of Plan-Structured Coding. BARA surpasses it at every budget, significantly in five of the six pairs, while incurring comparable or lower cost, reaching 59.0% against 55.6% at 9.56K versus 10.80K accounted tokens on Qwen3-30B at 20k, and 51.6% against 49.8% at 8.65K versus 10.19K on Qwen2.5-32B. The cost advantage over repeated sampling widens with the budget; for example, on Qwen2.5-32B it goes from parity at 10k to 1.5K fewer accounted tokens at 20k, as repeated sampling keeps drawing samples while BARA stops at the first success. As the budget loosens BARA again nears the budgeted oracle, trailing it by 1.9% on Qwen3-30B and 0.7% on Qwen2.5-32B at 20k. The same orchestration that aids function-level library tasks carries to a competitive-programming distribution, recovering most of the coverage the oracle attains there.

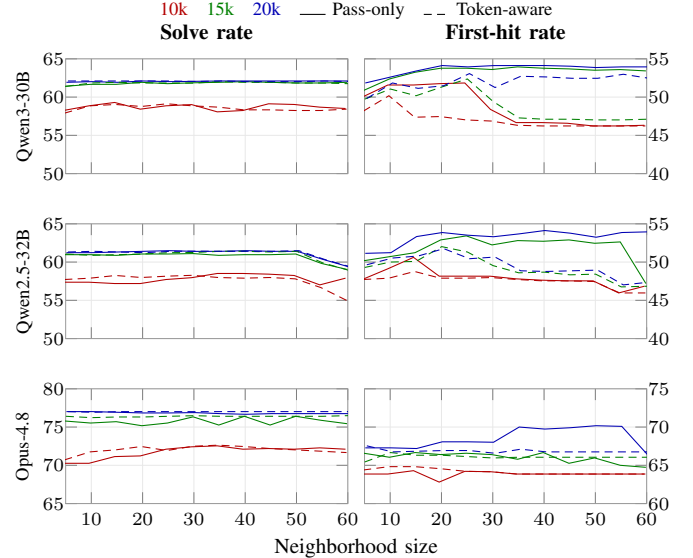


Fig. 3. Sensitivity of BARA to the neighborhood size k used for retrieving similar historical tasks. Rows correspond to models, columns to metrics, colors to task-level budgets, and line styles distinguish Pass-Only (PO) from Token-Aware (TA) feedback.

B. Sensitivity and Qualitative Analysis

a) *Neighborhood size:* We vary the ranker’s neighborhood k from 5 to 60 around the default $k = 30$ shown in Figure 3. Solve rate remains stable across the band $k = 25$ – 40 : within-setting variation is at most 1.1%, and at $k = 30$ solve rate lies within 0.6% of the best observed value in every setting. First-hit rate is more sensitive, varying by up to 5.3% within the same band, as small neighborhoods are dominated by sparse local evidence while large ones average out task-specific workflow differences. Final correctness is therefore robust around the default, whereas ranking remains more sensitive to local evidence; $k = 30$ lies at the center of the stable range and preserves near-optimal correctness without committing to a setting-specific first-hit optimum.

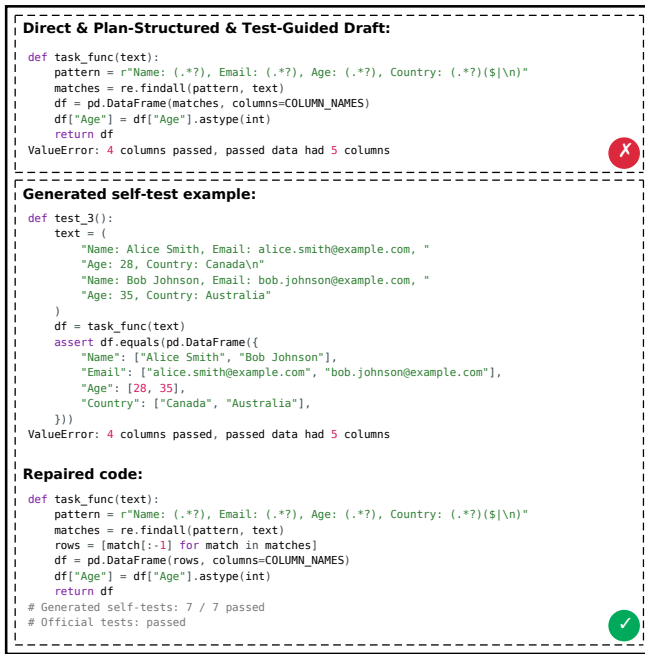


Fig. 4. Qualitative example of generated code under inference workflows.

b) *Qualitative case:* A Qwen2.5-32B case in Figure 4 demonstrates how generated self-tests localize a failure precisely enough to make repair a targeted edit driven by the localized defect. The task parses text records into a four-column pandas DataFrame. Both Direct Coding and Plan-Structured Coding produce a regex in which the trailing delimiter group is itself captured, yielding five fields per match instead of four and triggering the error *4 columns passed, passed data had 5 columns*. The plan in Plan-Structured Coding fixes the interface in advance but cannot anticipate this runtime mismatch, leaving the same error in place. Test-Guided Repair fails on the same draft, but its self-tests, each asserting an exact four-column DataFrame, localize the defect to the schema, not the parse: the records are extracted correctly, yet each match carries more fields than the column specification allows. The repair consequently retains the regex and modifies only the DataFrame construction to discard the delimiter group, restoring agreement with both the generated and official tests.

IX. THREATS TO VALIDITY

Construct validity. Solve rate is our primary metric, directly measuring the task objective of generating code that passes the benchmark tests. This operationalization omits readability, maintainability, and developer preference; accounted-token cost, first-hit rate, and capped-attempt rate complement it for efficiency and ranking behavior, while the subjective dimensions of code quality remain beyond our instrumentation.

Internal validity. The comparison could be confounded by differences in workflow implementation or token accounting. We use the same workflow pool, models, evaluator, token accounting, and budget protocol across all methods, leaving

the ranking and allocation strategies as the only varying components.

Conclusion validity. The gains remain contingent on the stability of workflow complementarity, since BARA converts complementarity into solve rate only where workflows retain non-identical solvable sets. The solved-task overlap shows a substantial workflow-specific region on every model, indicating the complementarity the method relies on is non-trivial.

External validity. BARA accumulates neighboring evidence online, and its advantage requires a task stream long enough for that evidence to form; the full-scale BigCodeBench and LiveCodeBench results indicate this holds at standard benchmark scale, confining the threat to markedly smaller settings. The gains further depend on the three-workflow pool studied here, and a different or larger workflow library may exhibit different complementarity, leaving broader libraries and task domains to future work.

X. CONCLUSION

This paper studied code generation as ranked workflow execution with task-level budget allocation and presented BARA, a framework that ranks Direct Coding, Plan-Structured Coding, and Test-Guided Repair for each task and allocates the task budget along the resulting sequence. The results show that multi-workflow orchestration improves code generation because the workflows overlap but still retain workflow-specific successes. BARA converts this complementarity into higher solve rate under a fixed budget, improving over the best individual workflow on all three models and outperforming repeated sampling in eight of nine model-budget settings. These gains generalize to LiveCodeBench, where BARA retains its advantage over both single workflows and repeated sampling.

The ablations identify how the gain arises. Task-conditioned ranking improves first-hit behavior by placing a successful workflow earlier in execution; adaptive allocation reduces capped attempts when the budget is binding by assigning caps from task-specific cost evidence; and token-aware feedback lowers cost while preserving solve rate by treating a cheap success as stronger evidence than an equally correct but budget-heavy one. These findings indicate that multi-workflow code generation depends less on which workflow is strongest than on how workflow order, per-attempt caps, and feedback design shape the fallback opportunities available under a shared task budget.

XI. DATA AND CODE AVAILABILITY

Code, results, prompts, and files used/generated for the study have been released in our replication package [68].

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [2] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–72, 2026.

- [3] M. A. Islam, M. E. Ali, and M. R. Parvez, "MapCoder: Multi-agent code generation for competitive problem solving," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 4912–4944. [Online]. Available: <https://aclanthology.org/2024.acl-long.269/>
- [4] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin, "CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 13 643–13 658. [Online]. Available: <https://aclanthology.org/2024.acl-long.737/>
- [5] M. A. Islam, M. E. Ali, and M. R. Parvez, "CodeSim: Multi-agent code generation and problem solving through simulation-driven planning and debugging," in *Findings of the Association for Computational Linguistics: NAACL 2025*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 5128–5154. [Online]. Available: <https://aclanthology.org/2025.findings-naacl.285/>
- [6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: agent-computer interfaces enable automated software engineering," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS '24. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [7] S. Cifani, M. L. Bernardi, and M. Cimitile, "Adaptive multimodal agents-based framework for automatic workflow execution," *arXiv preprint arXiv:2605.28607*, 2026.
- [8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying llm-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025.
- [9] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang *et al.*, "Aflow: Automating agentic workflow generation," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 34 040–34 077.
- [10] H. Chen, Z. Qi, H. Li, H. Chen, J. Chen, K. Peng, I. K. Kim, K. H. Lee, and S. He, "Beyond single llms: Enhanced code generation via multi-stage performance-guided llm orchestration," *arXiv preprint arXiv:2510.01379*, 2025.
- [11] Y. Li, L. Frampton, F. Mora, and E. Polgreen, "Online prompt selection for program synthesis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 11, 2025, pp. 11 282–11 289.
- [12] S. Arya, "Batched nonparametric bandits via k-nearest neighbor ucB," *arXiv preprint arXiv:2505.10498*, 2025.
- [13] Y. Xia, W. Ding, X.-D. Zhang, N. Yu, and T. Qin, "Budgeted bandit problems with continuous random costs," in *Asian conference on machine learning*. PMLR, 2016, pp. 317–332.
- [14] Y. Xia, T. Qin, W. Ma, N. Yu, and T.-Y. Liu, "Budgeted multi-armed bandits with multiple plays," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, ser. IJCAI'16. AAAI Press, 2016, p. 2210–2216.
- [15] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [16] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–23, 2025.
- [17] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "Codet: Code generation with generated tests," *arXiv preprint arXiv:2207.10397*, 2022.
- [18] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 8746–8825.
- [19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [20] T. Zheng, G. Zhang, T. Shen, X. Liu, B. Y. Lin, J. Fu, W. Chen, and X. Yue, "Opencodeinterpreter: Integrating code generation with execution and refinement," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 12 834–12 859.
- [21] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul *et al.*, "Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 66 602–66 656.
- [22] N. Jain, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "Livecodebench: Holistic and contamination free evaluation of large language models for code," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 58 791–58 831.
- [23] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [24] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [25] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.
- [26] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," *arXiv preprint arXiv:2204.05999*, 2022.
- [27] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.
- [28] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [29] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 27 168–27 188.
- [30] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: when the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [31] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang, and T. Yu, "Ds-1000: A natural and reliable benchmark for data science code generation," in *International Conference on Machine Learning*. PMLR, 2023, pp. 18 319–18 345.
- [32] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.
- [33] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *Advances in neural information processing systems*, vol. 36, pp. 11 809–11 822, 2023.
- [34] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.
- [35] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," in *International conference on machine learning*. PMLR, 2023, pp. 10 764–10 799.
- [36] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *arXiv preprint arXiv:2211.12588*, 2022.
- [37] A. Ni, S. Iyer, D. Radev, V. Stoyanov, W.-t. Yih, S. Wang, and X. V. Lin, "Lever: Learning to verify language-to-code generation with execution," in *International Conference on Machine Learning*. PMLR, 2023, pp. 26 106–26 128.
- [38] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.
- [39] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.
- [40] X. Yin, C. Ni, S. Wang, Z. Li, L. Zeng, and X. Yang, "Thinkrepair: Self-directed automated program repair," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1274–1286.
- [41] Y. Hu, Q. Zhou, Q. Chen, X. Li, L. Liu, D. Zhang, A. Kachroo, T. Oz, and O. Tripp, "Qualityflow: An agentic workflow for program synthesis

- controlled by llm quality checks,” *arXiv preprint arXiv:2501.17167*, 2025.
- [42] R. Zhu, B. Jiang, L. Mei, F. Yang, L. Wang, H. Gao, F. Bai, P. Zhao, Q. Lin, S. Rajmohan, and D. Zhang, “AdaptFlow: Adaptive workflow optimization via meta-learning,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 3287–3302. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.175/>
- [43] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [44] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. Lakshmanan, and A. H. Awadallah, “Hybrid llm: Cost-efficient and quality-aware query routing,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 41 348–41 366.
- [45] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “Routellm: Learning to route llms with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [46] W. Zhao, Y. Liu, Y. Wan, Y. Wang, Q. Wu, Z. Deng, J. Du, S. Liu, Y. Xu, and P. S. Yu, “knn-icl: Compositional task-oriented parsing generalization with nearest neighbor in-context learning,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 326–337.
- [47] Y. Gorishniy, I. Rubachev, N. Kartashev, D. Shlenskii, A. Kotelnikov, and A. Babenko, “Tabr: Tabular deep learning meets nearest neighbors,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 18 209–18 249.
- [48] H.-J. Ye, H.-H. Yin, D.-C. Zhan, and W.-L. Chao, “Revisiting nearest neighbor for tabular data: A deep tabular baseline two decades later,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 94 917–94 936.
- [49] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, “Large language monkeys: Scaling inference compute with repeated sampling,” *arXiv preprint arXiv:2407.21787*, 2024.
- [50] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling llm test-time compute optimally can be more effective than scaling model parameters,” *arXiv preprint arXiv:2408.03314*, 2024.
- [51] V.-R. Dănilă and C. Buiu, “A deep learning approach to censored regression,” *Pattern Anal. Appl.*, vol. 27, no. 1, Feb. 2024. [Online]. Available: <https://doi.org/10.1007/s10044-024-01216-9>
- [52] M. Sesia and V. Svetnik, “Doubly robust conformalized survival analysis with right-censored data,” *arXiv preprint arXiv:2412.09729*, 2024.
- [53] F. B. Hüttel, C. Riis, F. Rodrigues, and F. Pereira, “Bayesian active learning for censored regression,” in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2025, pp. 36–51.
- [54] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le *et al.*, “Least-to-most prompting enables complex reasoning in large language models,” *arXiv preprint arXiv:2205.10625*, 2022.
- [55] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023, pp. 2609–2634.
- [56] C. Lei, Y. Chang, N. Lipovetzky, and K. A. Ehinger, “Planning-driven programming: A large language model programming workflow,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 12 647–12 684. [Online]. Available: <https://aclanthology.org/2025.acl-long.621/>
- [57] X. Jiang, Y. Dong, L. Wang, Z. Fang, Q. Shang, G. Li, Z. Jin, and W. Jiao, “Self-planning code generation with large language models,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 7, pp. 1–30, 2024.
- [58] T. Ridnik, D. Kreda, and I. Friedman, “Code generation with alphacodium: From prompt engineering to flow engineering,” *arXiv preprint arXiv:2401.08500*, 2024.
- [59] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, “Enhancing llm-based code generation with complexity metrics: A feedback-driven approach,” in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2025, pp. 1416–1426.
- [60] S. Fan, X. Cong, Y. Fu, Z. Zhang, S. Zhang, Y. Liu, Y. Wu, Y. Lin, Z. Liu, and M. Sun, “Workflowllm: Enhancing workflow orchestration capability of large language models,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 24 498–24 525.
- [61] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, “Qwen2. 5-coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [62] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [63] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin *et al.*, “Qwen3 embedding: Advancing text embedding and reranking through foundation models,” *arXiv preprint arXiv:2506.05176*, 2025.
- [64] G. Yang, Y. Zhou, W. Cheng, X. Zhang, X. Chen, T. Y. Zhuo, K. Liu, X. Zhou, D. Lo, and T. Chen, “Less is more: Docstring compression in code generation,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–31, 2026.
- [65] J. Wang, “Token complexity theory for ai-augmented computing,” *arXiv preprint arXiv:2606.12647*, 2026.
- [66] S. Ren, X. Shen, Y. Zhou, A. Raj *et al.*, “Chinese language is not more efficient than english in vibe coding: A preliminary study on token cost and problem-solving rate,” *arXiv preprint arXiv:2604.14210*, 2026.
- [67] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.
- [68] Anonymous, “Replication package,” 2026. [Online]. Available: <https://anonymous.4open.science/r/code-gen-EB35>